МИНИСТЕРСТВО НАУКИ
И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное
образовательное учреждение высшего образования
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

НГТУ НЭТИ | Факультет прикладной математики и информатики

Кафедра прикладной математики

Лабораторная работа № 1

по дисциплине «Компьютерные технологии моделирования и анализа данных»

Векторный МКЭ

САЛЯЕВ АРТЁМ

ПММ-42 ЯНКОВСКИЙ ЕГОР

Преподаватели Кошкина Юлия Игоревна

Новосибирск, 2025

ЗАДАНИЕ

Построение и тестирование конечноэлементной вычислительной схемы с использованием векторных базисных функций на параллелепипедах и прямоугольниках.

1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1. МАТЕМАТИЧЕСКАЯ ПОСТАНОВКА

Решение задачи в трехмерных декартовых координатах, удовлетворяющей расчетной области Ω, будем искать из следующего уравнения:

$$\mathrm{rot}\frac{1}{\mu}\mathrm{rot}\overrightarrow{\mathbf{A}} + \gamma\overrightarrow{\mathbf{A}} = \overrightarrow{\mathbf{J}} \tag{1.1}$$

Эквивалентная вариационная постановка для уравнения (1.1) имеет вид:

$$\int\limits_{\Omega} \frac{1}{\mu}\mathrm{rot}\overrightarrow{\mathbf{A}}\,\mathrm{rot}\overrightarrow{\Psi}\,d\Omega + \int\limits_{\Omega} \gamma\overrightarrow{\mathbf{A}}\,\overrightarrow{\Psi}\,d\Omega = \int\limits_{\Omega} \overrightarrow{\mathbf{J}}\,\overrightarrow{\Psi}\,d\Omega. \tag{1.2}$$

1.2. ПРИНЦИПЫ ПОСТРОЕНИЯ ЛОКАЛЬНЫХ ВЕКТОРОВ, МАТРИЦ ЖЕСТКОСТИ И МАСС В ДВУМЕРНОМ ВЕКТОРНОМ МКЭ

Решение будем искать, используя билинейные базисные вектор-функции на прямоугольниках $\Omega_{rs} = [x_r, x_{r+1}] \times [y_s, y_{s+1}]$:

$$\overrightarrow{\psi}_1 = \begin{pmatrix} 0 \\ \frac{x_{r+1}-x}{h_x} \end{pmatrix}, \qquad \overrightarrow{\psi}_2 = \begin{pmatrix} 0 \\ \frac{x-x_r}{h_x} \end{pmatrix},$$

$$\overrightarrow{\psi}_3 = \begin{pmatrix} \frac{y_{s+1}-y}{h_y} \\ 0 \end{pmatrix}, \qquad \overrightarrow{\psi}_4 = \begin{pmatrix} \frac{y-y_s}{h_y} \\ 0 \end{pmatrix}.$$

где $h_x = x_{r+1} - x_r$, $h_y = y_{s+1} - y_s$.

Локальная матрица жёсткости $\hat{\mathbf{G}}$ на векторном прямоугольнике при $\overline{\mu} = const$ принимает вид:

$$\hat{\mathbf{G}} = \frac{1}{\overline{\mu}} \begin{pmatrix} \frac{h_y}{h_x} & -\frac{h_y}{h_x} & -1 & 1 \\ -\frac{h_y}{h_x} & \frac{h_y}{h_x} & 1 & -1 \\ -1 & 1 & \frac{h_x}{h_y} & -\frac{h_x}{h_y} \\ 1 & -1 & -\frac{h_x}{h_y} & \frac{h_x}{h_y} \end{pmatrix}.$$

Локальная матрица масс $\hat{\mathbf{M}}$ на векторном прямоугольнике при $\overline{\gamma} = const$ принимает вид:

$$\hat{\mathbf{M}} = \overline{\gamma}\frac{h_x h_y}{6} \begin{pmatrix} 2 & 1 & 0 & 0 \\ 1 & 2 & 0 & 0 \\ 0 & 0 & 2 & 1 \\ 0 & 0 & 1 & 2 \end{pmatrix}.$$

Локальный вектор правой части $\hat{\mathbf{b}}$ на векторном прямоугольнике принимает вид:

$$\hat{\mathbf{b}} = \hat{\mathbf{M}} \begin{pmatrix} F_y(x_r, y_{s+\frac{1}{2}}) \\ F_y(x_{r+1}, y_{s+\frac{1}{2}}) \\ F_x(x_{r+\frac{1}{2}}, y_s) \\ F_x(x_{r+\frac{1}{2}}, y_{s+1}) \end{pmatrix}.$$

1.3. ПРИНЦИПЫ ПОСТРОЕНИЯ ЛОКАЛЬНЫХ ВЕКТОРОВ, МАТРИЦ ЖЕСТКОСТИ И МАСС В ТРЕХМЕРНОМ ВЕКТОРНОМ МКЭ

Решение будем искать, используя билинейные базисные вектор-функции, задаваемые линейными функциями на параллелепипиде $\Omega_{rsp} = [x_r, x_{r+1}] \times [y_s, y_{s+1}] \times [z_p, z_{p+1}]$ следующего вида:

$$X_1(x) = \frac{x_{r+1} - x}{x_{r+1} - x_r}, \qquad X_2(x) = \frac{x - x_r}{x_{r+1} - x_r}, \tag{1.3}$$

$$Y_1(y) = \frac{y_{s+1} - y}{y_{s+1} - y_s}, \qquad Y_2(y) = \frac{y - y_s}{y_{s+1} - y_s}, \tag{1.4}$$

$$Z_1(z) = \frac{z_{p+1} - z}{z_{p+1} - z_p}, \qquad Z_2(z) = \frac{z - z_p}{z_{p+1} - z_p}. \tag{1.5}$$

Тогда базисная вектор-функция, ассоциированная с ребром $\{y = y_s, z = z_p\}$ имеет вид:

$$\overrightarrow{\psi}_1 = \begin{pmatrix} \frac{y_{s+1}-y}{h_y} \cdot \frac{z_{p+1}-z}{h_z} \\ 0 \\ 0 \end{pmatrix},$$

где $h_y = y_{s+1} - y_s$ $h_z = z_{p+1} - z_p$.

Полный список базисных вектор-функций на параллелепипиде $\Omega_{rsp} = [x_r, x_{r+1}] \times [y_s, y_{s+1}] \times [z_p, z_{p+1}]$ можно записать в виде:

$$\overrightarrow{\psi}_1 = \begin{pmatrix} Y_1 \cdot Z_1 \\ 0 \\ 0 \end{pmatrix}, \qquad \overrightarrow{\psi}_2 = \begin{pmatrix} Y_2 \cdot Z_1 \\ 0 \\ 0 \end{pmatrix}, \qquad \overrightarrow{\psi}_3 = \begin{pmatrix} Y_1 \cdot Z_2 \\ 0 \\ 0 \end{pmatrix},$$

$$\overrightarrow{\psi}_4 = \begin{pmatrix} Y_2 \cdot Z_2 \\ 0 \\ 0 \end{pmatrix}, \qquad \overrightarrow{\psi}_5 = \begin{pmatrix} 0 \\ X_1 \cdot Z_1 \\ 0 \end{pmatrix}, \qquad \overrightarrow{\psi}_6 = \begin{pmatrix} 0 \\ X_2 \cdot Z_1 \\ 0 \end{pmatrix},$$

$$\overrightarrow{\psi}_7 = \begin{pmatrix} 0 \\ X_1 \cdot Z_2 \\ 0 \end{pmatrix}, \qquad \overrightarrow{\psi}_8 = \begin{pmatrix} 0 \\ X_2 \cdot Z_2 \\ 0 \end{pmatrix}, \qquad \overrightarrow{\psi}_9 = \begin{pmatrix} Y_1 \cdot Z_2 \\ 0 \\ 0 \end{pmatrix},$$

$$\overrightarrow{\psi}_{10} = \begin{pmatrix} Y_1 \cdot Z_1 \\ 0 \\ 0 \end{pmatrix}, \qquad \overrightarrow{\psi}_{11} = \begin{pmatrix} Y_2 \cdot Z_1 \\ 0 \\ 0 \end{pmatrix}, \qquad \overrightarrow{\psi}_{12} = \begin{pmatrix} Y_1 \cdot Z_2 \\ 0 \\ 0 \end{pmatrix},$$

Формулы для вычисления глобальных матриц жёсткости $\mathbf{G}$ и масс $\mathbf{M}$, определяющих глобальную матрицу $\mathbf{C} = \mathbf{G} + \mathbf{M}$ конечноэлеметной СЛАУ имеют вид:

$$G_{ij} = \int\limits_{\Omega} \frac{1}{\mu} \mathrm{rot}\overrightarrow{\Psi}_i \cdot \mathrm{rot}\overrightarrow{\Psi}_j d\Omega, \qquad M_{ij} = \int\limits_{\Omega} \gamma \overrightarrow{\Psi}_i \cdot \overrightarrow{\Psi}_j d\Omega. \tag{1.6}$$

Компоненты глобального вектора $\mathbf{b}$ конечноэлементной СЛАУ определяются соотношением:

$$b_i = \int\limits_{\Omega} \overrightarrow{\mathbf{F}} \cdot \overrightarrow{\Psi}_i d\Omega. \tag{1.7}$$

Локальная матрица жёсткости $\hat{\mathbf{G}}$ на векторном параллелепипеде при $\overline{\mu} = const$ принимает вид:

$$\hat{\mathbf{G}} = \frac{1}{\overline{\mu}} \begin{pmatrix} \frac{h_x h_y}{6h_z}\mathbf{G}_1 + \frac{h_x h_z}{6h_y}\mathbf{G}_2 & -\frac{h_z}{6}\mathbf{G}_2 & \frac{h_y}{6}\mathbf{G}_3 \\ -\frac{h_z}{6}\mathbf{G}_2 & \frac{h_x h_y}{6h_z}\mathbf{G}_1 + \frac{h_y h_z}{6h_x}\mathbf{G}_2 & -\frac{h_x}{6}\mathbf{G}_1 \\ \frac{h_y}{6}\mathbf{G}_3^{\mathrm{T}} & -\frac{h_x}{6}\mathbf{G}_1 & \frac{h_x h_z}{6h_y}\mathbf{G}_1 + \frac{h_y h_z}{6h_x}\mathbf{G}_2 \end{pmatrix},$$

где

$$\mathbf{G}_1 = \begin{pmatrix} 2 & 1 & -2 & -1 \\ 1 & 2 & -1 & -2 \\ -2 & -1 & 2 & 1 \\ -1 & -2 & 1 & 2 \end{pmatrix}, \mathbf{G}_2 = \begin{pmatrix} 2 & -2 & 1 & -1 \\ -2 & 2 & -1 & 2 \\ 1 & -1 & 2 & -2 \\ -1 & 1 & -2 & 2 \end{pmatrix},$$

$$\mathbf{G}_3 = \begin{pmatrix} -2 & 2 & -1 & 1 \\ -1 & 1 & -2 & 2 \\ 2 & -2 & 1 & -1 \\ 1 & -1 & 2 & -2 \end{pmatrix}.$$

Матрицу $\hat{\mathbf{M}}$ можно представить в виде:

$$\mathbf{M} = \begin{pmatrix} \mathbf{D} & \mathbf{O} & \mathbf{O} \\ \mathbf{O} & \mathbf{D} & \mathbf{O} \\ \mathbf{O} & \mathbf{O} & \mathbf{D} \end{pmatrix},$$

где $\mathbf{O}$ - матрица 4×4, состоящая из нулей подматрица, а $\mathbf{D}$ определяется следующим образом:

$$\mathbf{D} = \begin{pmatrix} 4 & 2 & 2 & 1 \\ 2 & 4 & 1 & 2 \\ 2 & 1 & 4 & 2 \\ 1 & 2 & 2 & 4 \end{pmatrix}.$$

Локальный вектор правой части определяется в виде $\hat{\mathbf{b}}_i = \hat{\mathbf{M}} \cdot \hat{\mathbf{f}}_i$, где

$$\hat{\mathbf{f}}_i = \overrightarrow{\mathbf{F}}(\hat{x}_c, \hat{y}_c, \hat{z}_c) \cdot \overrightarrow{\mathbf{v}}/l, \tag{1.8}$$

$\hat{x}_c, \hat{y}_c, \hat{z}_c$ - координаты центра ребра, $\overrightarrow{\mathbf{v}}$ - вектор, направленный вдоль ребра Γ и сонаправленный с базисной вектор-функцией $\overrightarrow{\psi}_i$, l - длина данного вектора.

2. ИССЛЕДОВАНИЯ 2D

2.1. ПРЕДВАРИТЕЛЬНОЕ ОПИСАНИЕ

Проведем сначала тестирование разработанной программы по векторному МКЭ на работоспособность. Образец расчетной области изображен на рисунке 3.1. Это область $\Omega = [0.0, 2.0]_x \times [0.0, 2.0]_y$, она содержит 12 ребер, на всех границах будем задавать первые краевые условия. Середины наших элементов обозначены красным цветом, в этих точках мы будем рассматривать точность наших решений для оценки точности программы.

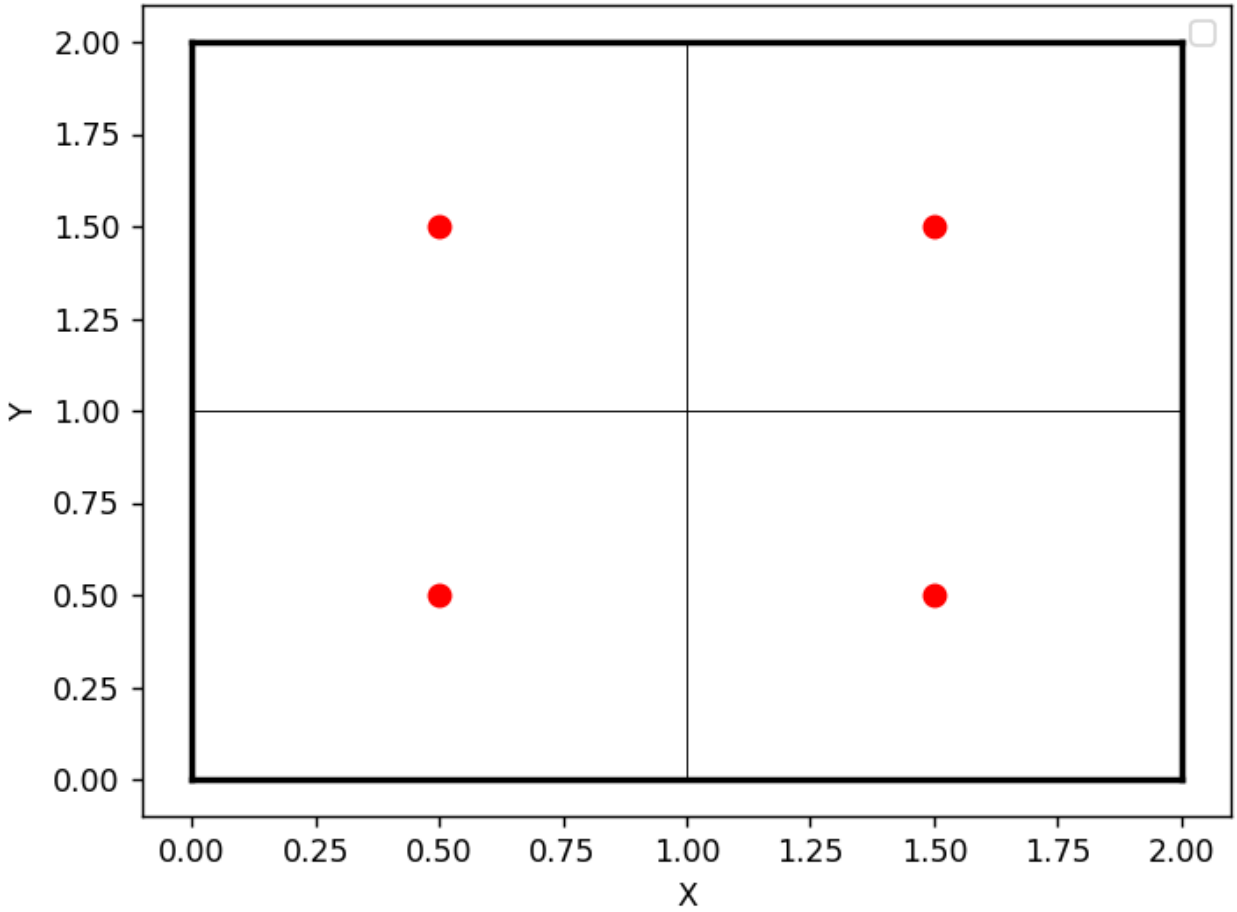

Рисунок 2.1 – Расчетная область

2.2. ТЕСТИРОВАНИЕ НА ПОРЯДОК АППРОКСИМАЦИИ

Тестирование будем проводить на дифференциальном уравнении (3.1) :

$$\mathrm{rot}\left(\frac{1}{\mu}\mathrm{rot}\overrightarrow{\mathbf{A}}\right)+\gamma\overrightarrow{\mathbf{A}}=\overrightarrow{\mathbf{F}} \tag{2.1}$$

Таблица 2.1 – Тестирование при $\overrightarrow{\mathbf{A}}=(1.0,1.0)^{\mathrm{T}}$, $\overrightarrow{\mathbf{F}}=(1.0,1.0)^{\mathrm{T}}$, $\mu=1$, $\gamma=1$

Точка	Значение	Абсолютная погрешность	Относительная погрешность
(0.5; 0.5)	1.00000000E+000; 1.00000000E+000	2.22044605E-016; 2.22044605E-016	2.22044605E-016; 2.22044605E-016
(1.5; 0.5)	1.00000000E+000; 1.00000000E+000	2.22044605E-016; 2.22044605E-016	2.22044605E-016; 2.22044605E-016
(0.5; 1.5)	1.00000000E+000; 1.00000000E+000	2.22044605E-016; 2.22044605E-016	2.22044605E-016; 2.22044605E-016
(1.5; 1.5)	1.00000000E+000; 1.00000000E+000	2.22044605E-016; 2.22044605E-016	2.22044605E-016; 2.22044605E-016

Таблица 2.2 – Тестирование при $\overrightarrow{\mathbf{A}}=(y,x)^{\mathrm{T}}$, $\overrightarrow{\mathbf{F}}=(y,x)^{\mathrm{T}}$, $\mu=1$, $\gamma=1$

Точка	Значение	Абсолютная погрешность	Относительная погрешность
(0.5; 0.5)	5.00000000E-001; 5.00000000E-001	0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000
(1.5; 0.5)	1.50000000E+000; 5.00000000E-001	0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000
(0.5; 1.5)	5.00000000E-001; 1.50000000E+000	0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000
(1.5; 1.5)	1.50000000E+000; 1.50000000E+000	0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000

Таблица 2.3 – Тестирование при $\overrightarrow{\mathbf{A}} = (1 + y + x; 1 + x)^{\mathrm{T}}$, $\overrightarrow{\mathbf{F}} = (1 + y + x; 1 + x)^{\mathrm{T}}$, $\mu = 1$, $\gamma = 1$

Точка	Значение	Абсолютная погрешность	Относительная погрешность
(0.5; 0.5)	2.00000000E+000; 1.50000000E+000	4.44089210E-016; 0.00000000E+000	2.22044605E-016; 0.00000000E+000
(1.5; 0.5)	3.00000000E+000; 2.50000000E+000	4.44089210E-016; 4.44089210E-016	1.48029737E-016; 1.77635684E-016
(0.5; 1.5)	3.00000000E+000; 1.50000000E+000	8.88178420E-016; 0.00000000E+000	2.96059473E-016; 0.00000000E+000
(1.5; 1.5)	4.00000000E+000; 2.50000000E+000	8.88178420E-016; 4.44089210E-016	2.22044605E-016; 1.77635684E-016

Таблица 2.4 – Тестирование при $\overrightarrow{\mathbf{A}} = (y^2; x^2)^{\mathrm{T}}$, $\overrightarrow{\mathbf{F}} = (y^2 - 2; x^2 - 2)^{\mathrm{T}}$, $\mu = 1$, $\gamma = 1$

Точка	Значение	Абсолютная погрешность	Относительная погрешность
(0.5; 0.5)	5.00000000E-001; 5.00000000E-001	2.50000000E-001; 2.50000000E-001	1.00000000E+000; 1.00000000E+000
(1.5; 0.5)	5.00000000E-001; 2.50000000E+000	2.50000000E-001; 2.50000000E-001	1.00000000E+000; 1.11111111E-001
(0.5; 1.5)	2.50000000E+000; 5.00000000E-001	2.50000000E-001; 2.50000000E-001	1.11111111E-001; 1.00000000E+000
(1.5; 1.5)	2.50000000E+000; 2.50000000E+000	2.50000000E-001; 2.50000000E-001	1.11111111E-001; 1.11111111E-001

Порядок аппроксимации равен 1.

2.3. ТЕСТИРОВАНИЕ НА ПОРЯДОК СХОДИМОСТИ

Таблица 2.5 – Тестирование при $\overrightarrow{\mathbf{A}} = (\mathrm{e}^y; \mathrm{e}^x)^{\mathrm{T}}$, $\overrightarrow{\mathbf{F}} = (0; 0)^{\mathrm{T}}$, $\mu = 1$, $\gamma = 1$

Точка	Относительная погрешность	Порядок $\log_2\left(\frac{\Delta i-1}{\Delta_i}\right)$
$(^4/_3;\ ^4/_3)$	1.0993868E-001 1.0993868E-001	—
	2.0790345E-002 2.0790345E-002	2.403
	5.7022633E-003 5.7022633E-003	1.866
	1.3475413E-003 1.3475413E-003	2.081
	3.4561228E-004 3.4561228E-004	1.963

Можем увидеть, что порядок сходимости равен 2.

3. ИССЛЕДОВАНИЯ 3D

3.1. ПРЕДВАРИТЕЛЬНОЕ ОПИСАНИЕ

Проведем сначала тестирование разработанной программы по векторному МКЭ на работоспособность. Образец расчетной области изображен на рисунке 3.1. Это область $\Omega = [0.0, 2.0]_x \times [0.0, 2.0]_y \times [0.0, 2.0]_z$, она содержит 54 ребра, на всех границах будем задавать первые краевые условия. Середины наших элементов обозначены красным цветом, в этих точках мы будем рассматривать точность наших решений для оценки точности программы.

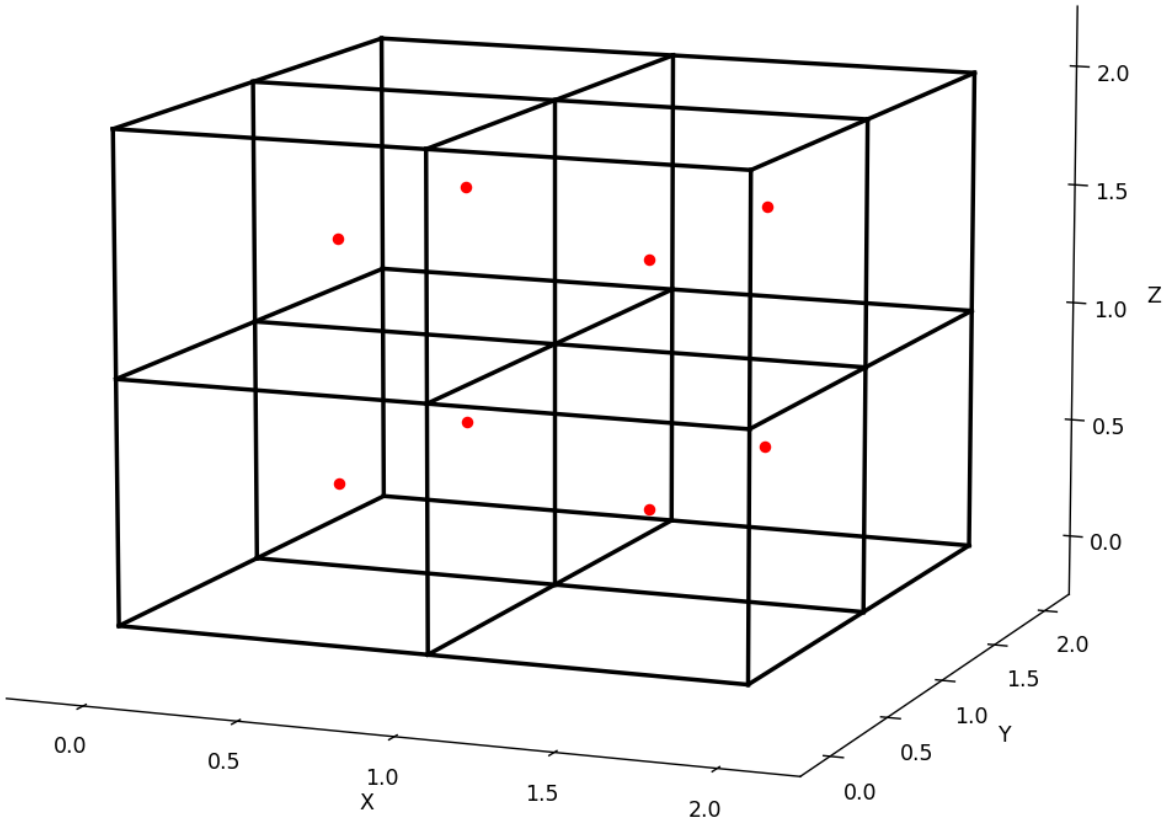

Рисунок 3.1 – Расчетная область

3.2. ТЕСТИРОВАНИЕ НА ПОРЯДОК АППРОКСИМАЦИИ

Тестирование будем проводить на дифференциальном уравнении (3.1) :

$$\mathrm{rot}\left(\frac{1}{\mu}\mathrm{rot}\overrightarrow{\mathbf{A}}\right) + \gamma\overrightarrow{\mathbf{A}} = \overrightarrow{\mathbf{F}} \tag{3.1}$$

Таблица 3.1 – Тестирование при $\overrightarrow{\mathbf{A}} = (1.0, 1.0, 1.0)^{\mathrm{T}}$, $\overrightarrow{\mathbf{F}} = (1.0, 1.0, 1.0)^{\mathrm{T}}$, $\mu = 1$, $\gamma = 1$

Точка	Значение	Абсолютная погрешность	Относительная погрешность
(0.5; 0.5; 0.5)	1.00000000E+000; 1.00000000E+000; 1.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 0.5; 0.5)	1.00000000E+000; 1.00000000E+000; 1.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 1.5; 0.5)	1.00000000E+000; 1.00000000E+000; 1.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 1.5; 0.5)	1.00000000E+000; 1.00000000E+000; 1.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 0.5; 1.5)	1.00000000E+000; 1.00000000E+000; 1.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 0.5; 1.5)	1.00000000E+000; 1.00000000E+000; 1.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 1.5; 1.5)	1.00000000E+000; 1.00000000E+000; 1.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 1.5; 1.5)	1.00000000E+000; 1.00000000E+000; 1.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000

Таблица 3.2 – Тестирование при $\overrightarrow{\mathbf{A}} = (y, z, x)^{\mathrm{T}}$, $\overrightarrow{\mathbf{F}} = (y, z, x)^{\mathrm{T}}$, $\mu = 1$, $\gamma = 1$

Точка	Значение	Абсолютная погрешность	Относительная погрешность
(0.5; 0.5; 0.5)	5.00000000E-001; 5.00000000E-001; 5.00000000E-001	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 0.5; 0.5)	5.00000000E-001; 5.00000000E-001; 1.50000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 1.5; 0.5)	1.50000000E+000; 5.00000000E-001; 5.00000000E-001	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 1.5; 0.5)	1.50000000E+000; 5.00000000E-001; 1.50000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 0.5; 1.5)	5.00000000E-001; 1.50000000E+000; 5.00000000E-001	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 0.5; 1.5)	5.00000000E-001; 1.50000000E+000; 1.50000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 1.5; 1.5)	1.50000000E+000; 1.50000000E+000; 5.00000000E-001	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 1.5; 1.5)	1.50000000E+000; 1.50000000E+000; 1.50000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000

Таблица 3.3 – Тестирование при $\overrightarrow{\mathbf{A}} = (1+y+x; 1+x+z; 1+x+y)^{\mathrm{T}}$, $\overrightarrow{\mathbf{F}} = (1+y+x; 1+x+z; 1+x+y)^{\mathrm{T}}$, $\mu = 1$, $\gamma = 1$

Точка	Значение	Абсолютная погрешность	Относительная погрешность
(0.5; 0.5; 0.5)	2.00000000E+000; 2.00000000E+000; 2.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 0.5; 0.5)	3.00000000E+000; 3.00000000E+000; 3.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 1.5; 0.5)	3.00000000E+000; 2.00000000E+000; 3.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 1.5; 0.5)	4.00000000E+000; 3.00000000E+000; 4.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 0.5; 1.5)	2.00000000E+000; 3.00000000E+000; 2.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 0.5; 1.5)	3.00000000E+000; 4.00000000E+000; 3.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 1.5; 1.5)	3.00000000E+000; 3.00000000E+000; 3.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 1.5; 1.5)	4.00000000E+000; 4.00000000E+000; 4.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000

Таблица 3.4 – Тестирование при $\overrightarrow{\mathbf{A}} = (y - z; x - z; x - y)^{\mathrm{T}}$, $\overrightarrow{\mathbf{F}} = (y - z; x - z; x - y)^{\mathrm{T}}$, $\mu = 1$, $\gamma = 1$

Точка	Значение	Абсолютная погрешность	Относительная погрешность
(0.5; 0.5; 0.5)	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 0.5; 0.5)	0.00000000E+000; 1.00000000E+000; 1.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 1.5; 0.5)	1.00000000E+000; 0.00000000E+000; -1.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 1.5; 0.5)	1.00000000E+000; 1.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 0.5; 1.5)	-1.00000000E+000; -1.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 0.5; 1.5)	-1.00000000E+000; 0.00000000E+000; 1.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 1.5; 1.5)	0.00000000E+000; -1.00000000E+000; -1.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 1.5; 1.5)	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000

Таблица 3.5 – Тестирование при $\overrightarrow{\mathbf{A}} = (y \cdot z; x \cdot z; x \cdot y)^{\mathrm{T}}$, $\overrightarrow{\mathbf{F}} = (y \cdot z; x \cdot z; x \cdot y)^{\mathrm{T}}$, $\mu = 1$, $\gamma = 1$

Точка	Значение	Абсолютная погрешность	Относительная погрешность
(0.5; 0.5; 0.5)	2.50000000E-001; 2.50000000E-001; 2.50000000E-001	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 0.5; 0.5)	2.50000000E-001; 7.50000000E-001; 7.50000000E-001	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 1.5; 0.5)	7.50000000E-001; 2.50000000E-001; 7.50000000E-001	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 1.5; 0.5)	7.50000000E-001; 7.50000000E-001; 2.25000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 0.5; 1.5)	7.50000000E-001; 7.50000000E-001; 2.500000000E-001	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 0.5; 1.5)	7.50000000E-001; 2.25000000E+000; 7.50000000E-001	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(0.5; 1.5; 1.5)	2.25000000E+000; 7.50000000E-001; 7.50000000E-001	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000
(1.5; 1.5; 1.5)	2.25000000E+000; 2.25000000E+000; 2.25000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000	0.00000000E+000; 0.00000000E+000; 0.00000000E+000

Таблица 3.6 – Тестирование при $\overrightarrow{\mathbf{A}} = (y^2; z^2; x^2)^{\mathrm{T}}$, $\overrightarrow{\mathbf{F}} = (y^2 - 2; z^2 - 2; x^2 - 2)^{\mathrm{T}}$, $\mu = 1$, $\gamma = 1$

Точка	Значение	Абсолютная погрешность	Относительная погрешность
(0.5; 0.5; 0.5)	5.00000000E-001; 5.00000000E-001; 5.00000000E-001	2.50000000E-001; 2.50000000E-001; 2.50000000E-001	1.00000000E+000; 1.00000000E+000; 1.00000000E+000
(1.5; 0.5; 0.5)	5.00000000E-001; 5.00000000E-001; 2.50000000E+000	2.50000000E-001; 2.50000000E-001; 2.50000000E-001	1.00000000E+000; 1.00000000E+000; 1.11111111E-001
(0.5; 1.5; 0.5)	2.50000000E+000; 5.00000000E-001; 5.00000000E-001	2.50000000E-001; 2.50000000E-001; 2.50000000E-001	1.111111111E-001; 1.00000000E+000; 1.00000000E+000
(1.5; 1.5; 0.5)	2.50000000E+000; 5.00000000E-001; 2.50000000E+000	2.50000000E-001; 2.50000000E-001; 2.50000000E-001	1.111111111E-001; 1.00000000E+000; 1.111111111E-001
(0.5; 0.5; 1.5)	5.00000000E-001; 2.50000000E+000; 5.000000000E-001	2.50000000E-001; 2.50000000E-001; 2.50000000E-001	1.00000000E+000; 1.111111111E-001; 1.00000000E+000
(1.5; 0.5; 1.5)	5.00000000E-001; 2.50000000E+000; 2.50000000E+000	2.50000000E-001; 2.50000000E-001; 2.50000000E-001	1.00000000E+000; 1.111111111E-001; 1.111111111E-001
(0.5; 1.5; 1.5)	2.50000000E+000; 2.50000000E+000; 5.00000000E-001	2.50000000E-001; 2.50000000E-001; 2.50000000E-001	1.111111111E-001; 1.111111111E-001; 1.00000000E+000
(1.5; 1.5; 1.5)	2.50000000E+000; 2.50000000E+000; 2.50000000E+000	2.50000000E-001; 2.50000000E-001; 2.50000000E-001	1.111111111E-001; 1.111111111E-001; 1.111111111E-001

Таблица 3.7 – Тестирование при $\overrightarrow{\mathbf{A}} = (y^2 + z^2; x^2 + z^2; x^2 + y^2)^{\mathrm{T}}$, $\overrightarrow{\mathbf{F}} = (y^2 + z^2 - 4; x^2 + z^2 - 4; x^2 + y^2 - 4)^{\mathrm{T}}$, $\mu = 1$, $\gamma = 1$

Точка	Значение	Абсолютная погрешность	Относительная погрешность
(0.5; 0.5; 0.5)	1.00000000E+000; 1.00000000E+000; 1.00000000E+000	5.00000000E-001; 5.00000000E-001; 5.00000000E-001	1.00000000E+000; 1.00000000E+000; 1.00000000E+000
(1.5; 0.5; 0.5)	1.00000000E+000; 3.00000000E+000; 3.00000000E+000	5.00000000E-001; 5.00000000E-001; 5.00000000E-001	1.00000000E+000; 2.00000000E-001; 2.00000000E-001
(0.5; 1.5; 0.5)	3.00000000E+000; 1.00000000E+000; 3.00000000E+000	5.00000000E-001; 5.00000000E-001; 5.00000000E-001	2.00000000E-001; 1.00000000E+000; 2.00000000E-001
(1.5; 1.5; 0.5)	3.00000000E+000; 3.00000000E+000; 5.00000000E+000	5.00000000E-001; 5.00000000E-001; 5.00000000E-001	2.00000000E-001; 2.00000000E-001; 1.111111111E-001
(0.5; 0.5; 1.5)	3.00000000E+000; 3.00000000E+000; 1.00000000E+000	5.00000000E-001; 5.00000000E-001; 5.00000000E-001	2.00000000E-001; 2.00000000E-001; 1.00000000E+000
(1.5; 0.5; 1.5)	3.00000000E+000; 5.00000000E+000; 3.00000000E+000	5.00000000E-001; 5.00000000E-001; 5.00000000E-001	2.00000000E-001; 1.111111111E-001; 2.00000000E-001
(0.5; 1.5; 1.5)	5.00000000E+000; 3.00000000E+000; 3.00000000E+000	5.00000000E-001; 5.00000000E-001; 5.00000000E-001	1.111111111E-001; 2.00000000E-001; 2.00000000E-001
(1.5; 1.5; 1.5)	5.00000000E+000; 5.00000000E+000; 5.00000000E+000	5.00000000E-001; 5.00000000E-001; 5.00000000E-001	1.111111111E-001; 1.111111111E-001; 1.111111111E-001

Порядок аппроксимации равен 1.

3.3. ТЕСТИРОВАНИЕ НА ПОРЯДОК СХОДИМОСТИ

Таблица 3.8 – Тестирование при $\overrightarrow{\mathbf{A}} = (\mathrm{e}^y; \mathrm{e}^z; \mathrm{e}^x)^{\mathrm{T}}$, $\overrightarrow{\mathbf{F}} = (0; 0; 0)^{\mathrm{T}}$, $\mu = 1$, $\gamma = 1$

Точка	Относительная погрешность	Порядок $\log_2\left(\frac{\Delta i-1}{\Delta_i}\right)$
$(^4/_3;\ ^4/_3;\ ^4/_3)$	1.1722162E-001 1.1722162E-001 1.1722162E-001	—
	2.2667516E-002 2.2667516E-002 2.2667516E-002	2.278
	6.1959597E-003 6.1959597E-003 6.1959597E-003	1.871
	1.4717991E-003 1.4717991E-003 1.4717991E-003	2.074
	3.7673935E-004 3.7673935E-004 3.7673935E-004	1.966

Можем увидеть, что порядок сходимости стремится к 2.

СПИСОК ЛИТЕРАТУРЫ

1. Ю.Г. Соловейчик, М.Э. Рояк, М.Г. Персова Метод конечных элементов для скалярных и векторных задач Учеб. пособие. — Новосибирск: Изд-во НГТУ, 2007 — 896 с.

2. А.Н. Тихонов, А.А. Самарский Уравнения математической физики: Учеб.пособие. / А.Н. Тихонов, А.А. Самарский — 6-е изд., — М: Изд-во МГУ, 1999 — 799 с.

ПРИЛОЖЕНИЕ. ТЕКСТ ПРОГРАММЫ.

Program.cs

```
#define TESTING
// RELEASE
// TESTING
#define NODRAWING
#define SOLVE2DIM
using Project;
using System.Globalization;
using Solver;
using Manager;
using System.Numerics;
using Processor;
using Solution;
using MathObjects;
using Functions;
using Grid;
using DataStructs;


CultureInfo.CurrentCulture = CultureInfo.InvariantCulture;

string BordersInfo = Path.GetFullPath("../../../../Data/Input/Borders.dat");
string AnswerPath = Path.GetFullPath("../../../../Data/Output/");
string TimePath = Path.GetFullPath("../../../../Data/Input/Time.dat");

FEM3D myFEM3D_test = new();
myFEM3D_test.ConstructMesh(3, 3, 3);
myFEM3D_test.GenerateArrays();
myFEM3D_test.ConstructMatrixAndVector();
myFEM3D_test.SetSolver(new LOS());
myFEM3D_test.Solve();
myFEM3D_test.TestOutput(AnswerPath);
myFEM3D_test.WriteData(AnswerPath);

return 0;
```

LocalMatrix2D.cs

```
namespace MathObjects;

public class LocalMatrixG2D (double mu, double hx, double hy) : Matrix
{
    private readonly double _mu = mu;
    private readonly double _hx = hx;
    private readonly double _hy = hy;

    public override double this[int i, int j]
    {
        get
        {
            return (i % 4, j % 4)
            switch
            {
                (0, 0) or (1, 1) => _hy / (_hx * _mu),
                (0, 1) or (1, 0) => -1.0D * _hy / (_mu * _hx),
                (0, 2) or (1, 3) or (2, 0) or (3, 1) => -1.0D / _mu,
                (0, 3) or (1, 2) or (2, 1) or (3, 0) =>  1.0D / _mu,
                (2, 2) or (3, 3) => _hx / (_hy * _mu),
                (2, 3) or (3, 2) => -1.0D * _hx / (_mu * _hy),
                _ => throw new ArgumentOutOfRangeException("Out of local matrix 2d
                ↪  range"),
            };
        }
        set{}
    }
}

public class LocalMatrixM2D(double gamma, double hx, double hy) : Matrix
{
    private readonly double _gamma = gamma;
    private readonly double _hx = hx;
    private readonly double _hy = hy;

    private readonly double[,] D = {{2.0D, 1.0D, 0.0D, 0.0D},
                                    {1.0D, 2.0D, 0.0D, 0.0D},
                                    {0.0D, 0.0D, 2.0D, 1.0D},
```

```
                                    {0.0D, 0.0D, 1.0D, 2.0D}};

    public override double this[int i, int j]
    {
        get => _gamma * _hx * _hy * D[i, j] / 6.0D;
        set{}
    }
}
```

LocalVector2D.cs

```
using static Functions.Function;

namespace MathObjects;


public class LocalVector2D : Vector
{
    private readonly double x0;
    private readonly double x1;
    private readonly double y0;
    private readonly double y1;
    private readonly double xm;
    private readonly double ym;
    private readonly double t;

    private LocalMatrixM2D M;

    public override double this[int i]
    {
        get
        {
            static double ScalarMult((double, double) a, (double, double) b)
            => a.Item1 * b.Item1 + a.Item2 * b.Item2;

            (double, double) Divide ((double, double) v, double sc)
            => (v.Item1 / sc, v.Item2 / sc);

            double Length((double, double) v)
            => Math.Sqrt(Math.Pow(v.Item1, 2) + Math.Pow(v.Item2, 2));
```

```
30
31              // Очень спорно.
32              List<double> vect = [
33                  ScalarMult(F(x0, ym, t), (0.0D, 1.0D)),
34                  ScalarMult(F(x1, ym, t), (0.0D, 1.0D)),
35                  ScalarMult(F(xm, y0, t), (1.0D, 0.0D)),
36                  ScalarMult(F(xm, y1, t), (1.0D, 0.0D))
37              ];
38
39              double ans = 0.0D;
40              for (int j = 0; j < vect.Count; j++)
41                  ans += M[i, j] * vect[j];
42              return ans;
43          }
44      }
45
46      public LocalVector2D(double x0, double x1, double y0, double y1, double t)
47      {
48          this.x0 = x0;
49          this.x1 = x1;
50          this.y0 = y0;
51          this.y1 = y1;
52          this.t = t;
53          xm = 0.5D * (x1 + x0);
54          ym = 0.5D * (y1 + y0);
55          M = new(1.0, x1 - x0, y1 - y0);
56          Generate();
57      }
58
59      private void Generate()
60      {
61          static double ScalarMult((double, double) a, (double, double) b)
62          => a.Item1 * b.Item1 + a.Item2 * b.Item2;
63
64          List<double> vect = [
65              ScalarMult(F(x0, ym, t), (0.0D, 1.0D)),
66              ScalarMult(F(x1, ym, t), (0.0D, 1.0D)),
67              ScalarMult(F(xm, y0, t), (1.0D, 0.0D)),
68              ScalarMult(F(xm, y1, t), (1.0D, 0.0D)),
69          ];
```

```
70
71          double ans = 0.0D;
72          for (int i = 0; i < vect.Count; i++)
73          {
74              for (int j = 0; j < vect.Count; j++)
75                  ans += M[i, j] * vect[j];
76              ans = 0.0D;
77          }
78      }
79  }
```

FEM2D.cs

```
1   namespace Project;
2
3   using System.Collections.Immutable;
4   using System.Numerics;
5   using MathObjects;
6   using Solver;
7   using Grid;
8   using DataStructs;
9   using System.Diagnostics;
10  using Functions;
11  using System.ComponentModel.DataAnnotations;
12  using System.Timers;
13  using System;
14
15  public class FEM2D : FEM
16  {
17
18      public ArrayOfRibs ribsArr;
19
20      public FEM2D(Mesh2Dim mesh)
21      {
22          mesh2D = mesh;
23
24      }
25
26      public void GenerateArrays()
27      {
```

```
28          timeMesh = [1.0D];
29          if (mesh2D is null) throw new ArgumentNullException("mesh is null!");
30          pointsArr = MeshGenerator.GenerateListOfPoints(mesh2D);
31          ribsArr = MeshGenerator.GenerateListOfRibs(mesh2D, pointsArr);
32          elemsArr = MeshGenerator.GenerateListOfElems2D(ref mesh2D);
33          bordersArr = MeshGenerator.GenerateListOfBorders(mesh2D);
34      }
35
36      public void ConstructMatrixAndVector()
37      {
38          if (elemsArr is null) throw new ArgumentNullException("elemsArr is null");
39          if (bordersArr is null) throw new ArgumentNullException("bordersArr is null");
40
41          var sparceMatrix = new GlobalMatrix(ribsArr.Count);
42          Generator.BuildPortait(ref sparceMatrix, ribsArr.Count, elemsArr);
43
44          var G = new GlobalMatrix(sparceMatrix);
45          Generator.FillMatrixG2D(ref G, ribsArr, elemsArr);
46
47          var M = new GlobalMatrix(sparceMatrix);
48          Generator.FillMatrixM2D(ref M, ribsArr, elemsArr);
49
50          Matrix = G + M;
51
52          var b = new GlobalVector(ribsArr.Count);
53          Generator.FillVector2D(ref b, ribsArr, elemsArr, 0.0);
54          Vector = b;
55
56          Generator.ConsiderBoundaryConditions2D(ref Matrix, ref Vector, ribsArr,
            ↪  bordersArr, 0.0D);
57      }
58
59      public void Solve()
60      {
61          if (solver is null) throw new ArgumentNullException("Solver is null");
62          if (Matrix is null) throw new ArgumentNullException("Matrix is null");
63          if (Vector is null) throw new ArgumentNullException("Vector is null");
64          Solutions = new GlobalVector[timeMesh.Length];
65          Discrepancy = new GlobalVector[timeMesh.Length];
66          (Solutions[0], Discrepancy[0]) = solver.Solve(Matrix, Vector);
```

```
67          //
68          //if (timeMesh.Length > 1)
69          //{
70          //    for (int i = 0; i < timeMesh.Length; i++)
71          //    {
72          //        if (i == 1 || i == 0)
73          //        {
74          //            Solutions[i] = new GlobalVector(ribsArr.Count);
75          //            for (int j = 0; j < ribsArr.Count; j++)
76          //            {
77          //                var antinormal = ((ribsArr[j].b.X - ribsArr[j].a.X) /
    ↪ ribsArr[j].Length,
78          //                                  (ribsArr[j].b.Y - ribsArr[j].a.Y) /
    ↪ ribsArr[j].Length,
79          //                                  (ribsArr[j].b.Z - ribsArr[j].a.Z) /
    ↪ ribsArr[j].Length);
80          //                var pointat = ((ribsArr[j].b.X + ribsArr[j].a.X) / 2.0D,
81          //                               (ribsArr[j].b.Y + ribsArr[j].a.Y) / 2.0D,
82          //                               (ribsArr[j].b.Z + ribsArr[j].a.Z) / 2.0D);
83          //
84          //                var valueat = Function.A(pointat.Item1, pointat.Item2,
    ↪ pointat.Item3, timeMesh[i]);
85          //                Solutions[i][j] = valueat.Item1 * antinormal.Item1 +
    ↪ valueat.Item2 * antinormal.Item2 + valueat.Item3 * antinormal.Item3;
86          //            }
87          //            continue;
88          //        }
89          //        double t0 = timeMesh[i];
90          //        double t1 = timeMesh[i - 1];
91          //        double t2 = timeMesh[i - 2];
92          //
93          //        double deltT = t0 - t2;
94          //        double deltT0 = t0 - t1;
95          //        double deltT1 = t1 - t2;
96          //
97          //        double tau0 = (deltT + deltT0) / (deltT * deltT0);
98          //        double tau1 = deltT / (deltT1 * deltT0);
99          //        double tau2 = deltT0 / (deltT * deltT1);
100         //
101         //        var sparceMatrix = new GlobalMatrix(ribsArr.Count);
```

```
102         //        Generator.BuildPortait(ref sparceMatrix, ribsArr.Count, elemsArr);
103
104         //        var G = new GlobalMatrix(sparceMatrix);
105         //        Generator.FillMatrixG(ref G, ribsArr, elemsArr);
106         //
107         //        var M = new GlobalMatrix(sparceMatrix);
108         //        Generator.FillMatrixM(ref M, ribsArr, elemsArr);
109         //
110         //        Matrix = G + M + tau0 * M;
111
112         //        var b = new GlobalVector(ribsArr.Count);
113         //        Generator.FillVector3D(ref b, ribsArr, elemsArr, timeMesh[i]);
114         //        Vector = b - tau2 * M * Solutions[i - 2] + tau1 * M * Solutions[i - 1];
115
116         //        Generator.ConsiderBoundaryConditions(ref Matrix, ref Vector, ribsArr,
        ↪  bordersArr, timeMesh[i]);
117         //        (Solutions[i], Discrepancy[i]) = solver.Solve(Matrix, Vector);
118         //    }
119         //}
120     }
121
122     public void WriteData2D(string path)
123     {
124         if (Solutions is null) throw new ArgumentNullException("No solutions");
125
126         for (int t = 0; t < timeMesh.Length; t++)
127         {
128             using var sw = new StreamWriter(path + $"/Answer_{timeMesh[t]}.txt");
129             for (int i = 0; i < Solutions[t].Size; i++)
130                 if (i == 16 || i == 24 || i == 26 || i == 27 || i == 29 || i == 37)
131                     sw.WriteLine($"{i} {Solutions[t][i]:E8}");
132             sw.Close();
133         }
134     }
135
136     public void WriteData(string path)
137     {
138         if (Solutions is null) throw new ArgumentNullException("No solutions");
139
140         for (int t = 0; t < timeMesh.Length; t++)
```

```
141         {
142             using var sw = new StreamWriter(path +
                ↪ $"/A_phi/Answer3D/Answer_{timeMesh[t]}.txt");
143             for (int i = 0; i < Solutions[t].Size; i++)
144                 if (i == 16 || i == 24 || i == 26 || i == 27 || i == 29 || i == 37)
145                     sw.WriteLine($"{i} {Solutions[t][i]:E8}");
146             sw.Close();
147         }
148     }
149
150     public void TestPoint(double x, double y)
151     {
152         Point testPoint = new(x, y);
153
154         if (mesh2D.nodesX[0] <= x && x <= mesh2D.nodesX[^1] &&
155             mesh2D.nodesY[0] <= y && y <= mesh2D.nodesY[^1])
156         {
157             foreach (var elem in elemsArr)
158             {
159                 int[] elem_local = [elem[1], elem[2],
160                                     elem[0], elem[3]];
161
162                 var ribX = ribsArr[elem_local[2]];
163                 var ribY = ribsArr[elem_local[0]];
164
165                 // if inside local elem.
166                 if (ribX.a.X <= x && x <= ribX.b.X &&
167                     ribY.a.Y <= y && y <= ribY.b.Y)
168                 {
169                     var eps = (x - ribX.a.X) / (ribX.b.X - ribX.a.X);
170                     var nu = (y - ribY.a.Y) / (ribY.b.Y - ribY.a.Y);
171
172
173                     double[] q = [Solutions[0][elem_local[0]],
                        ↪ Solutions[0][elem_local[1]],
174                                   Solutions[0][elem_local[2]],
                                      ↪ Solutions[0][elem_local[3]]];
175
176
177                     var ans = BasisFunctions2DVec.GetValue(eps, nu, q);
```

```
178                     var theorValue = Function.A(x, y, 0.0D);
179
180                     Console.WriteLine($"FEM A  {ans.Item1:E15} {ans.Item2:E15}");
181                     Console.WriteLine($"Theor  {theorValue.Item1:E15}
                        ↪  {theorValue.Item2:E15}");
182
183                     var currAbsDiscX = Math.Abs(ans.Item1 - theorValue.Item1);
184                     var currAbsDiscY = Math.Abs(ans.Item2 - theorValue.Item2);
185
186                     var currRelDiscX = currAbsDiscX / Math.Abs(theorValue.Item1);
187                     var currRelDiscY = currAbsDiscY / Math.Abs(theorValue.Item2);
188
189                     Console.WriteLine($"CurrAD {currAbsDiscX:E15} {currAbsDiscY:E15}");
190                     Console.WriteLine($"CurrRD {currRelDiscX:E15} {currRelDiscY:E15}\n");
191                     break;
192                 }
193             }
194         }
195     }
196
197     public void TestPoint(double x, double y, double z)
198     {
199         Point testPoint = new(x, y, z);
200
201         if (mesh.nodesX[0] <= x && x <= mesh.nodesX[^1] &&
202             mesh.nodesY[0] <= y && y <= mesh.nodesY[^1] &&
203             mesh.nodesZ[0] <= z && z <= mesh.nodesZ[^1])
204         {
205             var absDiscX = 0.0D;
206             var absDiscY = 0.0D;
207             var absDiscZ = 0.0D;
208
209             var absDivX = 0.0D;
210             var absDivY = 0.0D;
211             var absDivZ = 0.0D;
212
213             var relDiscX = 0.0D;
214             var relDiscY = 0.0D;
215             var relDiscZ = 0.0D;
216
```

```
217             var relDivX = 0.0D;
218             var relDivY = 0.0D;
219             var relDivZ = 0.0D;
220
221             foreach (var elem in elemsArr)
222             {
223                 int[] elem_local = [elem[0], elem[3], elem[8], elem[11],
224                                     elem[1], elem[2], elem[9], elem[10],
225                                     elem[4], elem[5], elem[6], elem[7]];
226
227                 var ribX = ribsArr[elem_local[0]];
228                 var ribY = ribsArr[elem_local[4]];
229                 var ribZ = ribsArr[elem_local[8]];
230
231                 // if inside local elem.
232                 if (ribX.a.X <= x && x <= ribX.b.X &&
233                     ribY.a.Y <= y && y <= ribY.b.Y &&
234                     ribZ.a.Z <= z && z <= ribZ.b.Z)
235                 {
236                     var eps = (x - ribX.a.X) / (ribX.b.X - ribX.a.X);
237                     var nu = (y - ribY.a.Y) / (ribY.b.Y - ribY.a.Y);
238                     var khi = (z - ribZ.a.Z) / (ribZ.b.Z - ribZ.a.Z);
239
240
241                     double[] q = [Solutions[0][elem_local[0]],
                    ↪  Solutions[0][elem_local[1]], Solutions[0][elem_local[2]],
                    ↪  Solutions[0][elem_local[3]],
242                                 Solutions[0][elem_local[4]],
                                    ↪  Solutions[0][elem_local[5]],
                                    ↪  Solutions[0][elem_local[6]],
                                    ↪  Solutions[0][elem_local[7]],
243                                 Solutions[0][elem_local[8]],
                                    ↪  Solutions[0][elem_local[9]],
                                    ↪  Solutions[0][elem_local[10]],
                                    ↪  Solutions[0][elem_local[11]]];
244
245
246                     var ans = BasisFunctions3DVec.GetValue(eps, nu, khi, q);
247                     var theorValue = Function.A(x, y, z, 0.0D);
248
```

```
249                     Console.WriteLine($"FEM A  {ans.Item1:E15} {ans.Item2:E15}
                        ↪  {ans.Item3:E15}");
250                     Console.WriteLine($"Theor  {theorValue.Item1:E15}
                        ↪  {theorValue.Item2:E15} {theorValue.Item3:E15}");
251
252                     var currAbsDiscX = Math.Abs(ans.Item1 - theorValue.Item1);
253                     var currAbsDiscY = Math.Abs(ans.Item2 - theorValue.Item2);
254                     var currAbsDiscZ = Math.Abs(ans.Item3 - theorValue.Item3);
255
256                     var currRelDiscX = currAbsDiscX / Math.Abs(theorValue.Item1);
257                     var currRelDiscY = currAbsDiscY / Math.Abs(theorValue.Item2);
258                     var currRelDiscZ = currAbsDiscZ / Math.Abs(theorValue.Item3);
259
260                     Console.WriteLine($"CurrAD {currAbsDiscX:E15} {currAbsDiscY:E15}
                        ↪  {currAbsDiscZ:E15}");
261                     Console.WriteLine($"CurrRD {currRelDiscX:E15} {currRelDiscY:E15}
                        ↪  {currRelDiscZ:E15}\n");
262
263                     break;
264                 }
265             }
266         }
267     }
268
269 public void TestOutput2D(string path)
270     {
271         using var sw = new StreamWriter(path + "/Answer_Test.txt");
272
273         var absDiscX = 0.0D;
274         var absDiscY = 0.0D;
275
276         var absDivX = 0.0D;
277         var absDivY = 0.0D;
278
279         var relDiscX = 0.0D;
280         var relDiscY = 0.0D;
281
282         var relDivX = 0.0D;
283         var relDivY = 0.0D;
284
```

```
285         var squareDiffX = 0.0D;
286         var squareDiffY = 0.0D;
287 
288         int iter = 0;
289 
290         foreach (var elem in elemsArr)
291         {
292             int[] elem_local = [elem[1], elem[2],
293                                 elem[0], elem[3]];
294 
295             var x = 0.5D * (ribsArr[elem_local[2]].a.X + ribsArr[elem_local[2]].b.X);
296             var y = 0.5D * (ribsArr[elem_local[0]].a.Y + ribsArr[elem_local[0]].b.Y);
297 
298             sw.WriteLine($"Points {x:E15} {y:E15}");
299 
300             var eps = (x - ribsArr[elem_local[2]].a.X) / (ribsArr[elem_local[2]].b.X -
                ↪  ribsArr[elem_local[2]].a.X);
301             var nu = (y - ribsArr[elem_local[0]].a.Y) / (ribsArr[elem_local[0]].b.Y -
                ↪  ribsArr[elem_local[0]].a.Y);
302 
303             double[] q = [Solutions[0][elem_local[0]], Solutions[0][elem_local[1]],
304                           Solutions[0][elem_local[2]], Solutions[0][elem_local[3]]];
305 
306             var ans = BasisFunctions2DVec.GetValue(eps, nu, q);
307             var theorValue = Function.A(x, y, 0.0D);
308 
309             sw.WriteLine($"FEM A  {ans.Item1:E15} {ans.Item2:E15}");
310             sw.WriteLine($"Theor  {theorValue.Item1:E15} {theorValue.Item2:E15}");
311 
312             var currAbsDiscX = Math.Abs(ans.Item1 - theorValue.Item1);
313             var currAbsDiscY = Math.Abs(ans.Item2 - theorValue.Item2);
314 
315             squareDiffX += currAbsDiscX * currAbsDiscX;
316             squareDiffY += currAbsDiscY * currAbsDiscY;
317 
318             var currRelDiscX = currAbsDiscX / Math.Abs(theorValue.Item1);
319             var currRelDiscY = currAbsDiscY / Math.Abs(theorValue.Item2);
320 
321             sw.WriteLine($"CurrAD {currAbsDiscX:E15} {currAbsDiscY:E15}");
322             sw.WriteLine($"CurrRD {currRelDiscX:E15} {currRelDiscY:E15}\n");
```

```
323
324             absDiscX += currAbsDiscX;
325             absDiscY += currAbsDiscY;
326
327             absDivX += theorValue.Item1;
328             absDivY += theorValue.Item2;
329
330             relDiscX += currRelDiscX * currRelDiscX;
331             relDiscY += currRelDiscY * currRelDiscY;
332
333             relDivX += theorValue.Item1 * theorValue.Item1;
334             relDivY += theorValue.Item2 * theorValue.Item2;
335
336             iter++;
337         }
338         sw.WriteLine($"Avg disc: {absDiscX / iter:E15} {absDiscY / iter:E15}");
339         sw.WriteLine($"Rel disc: {Math.Sqrt(relDiscX / relDivX):E15} {Math.Sqrt(relDiscY
    ↪  / relDivY):E15}");
340         sw.WriteLine($"SKO: {Math.Sqrt(squareDiffX / elemsArr.Length):E15}
    ↪  {Math.Sqrt(squareDiffY / elemsArr.Length):E15}");
341         sw.Close();
342     }
343
344
345     public void TestOutput(string path)
346     {
347         using var sw = new StreamWriter(path + "/A_phi/Answer3D/Answer_Test.txt");
348
349         var absDiscX = 0.0D;
350         var absDiscY = 0.0D;
351         var absDiscZ = 0.0D;
352
353         var absDivX = 0.0D;
354         var absDivY = 0.0D;
355         var absDivZ = 0.0D;
356
357         var relDiscX = 0.0D;
358         var relDiscY = 0.0D;
359         var relDiscZ = 0.0D;
360
```

```
361         var relDivX = 0.0D;
362         var relDivY = 0.0D;
363         var relDivZ = 0.0D;
364
365         var squareDiffX = 0.0D;
366         var squareDiffY = 0.0D;
367         var squareDiffZ = 0.0D;
368
369         int iter = 0;
370
371         foreach (var elem in elemsArr)
372         {
373             int[] elem_local = [elem[0], elem[3], elem[8], elem[11],
374                                 elem[1], elem[2], elem[9], elem[10],
375                                 elem[4], elem[5], elem[6], elem[7]];
376
377             var x = 0.5D * (ribsArr[elem_local[0]].a.X + ribsArr[elem_local[0]].b.X);
378             var y = 0.5D * (ribsArr[elem_local[4]].a.Y + ribsArr[elem_local[4]].b.Y);
379             var z = 0.5D * (ribsArr[elem_local[8]].a.Z + ribsArr[elem_local[8]].b.Z);
380
381             sw.WriteLine($"Points {x:E15} {y:E15} {z:E15}");
382
383             var eps = (x - ribsArr[elem_local[0]].a.X) / (ribsArr[elem_local[0]].b.X -
                ↪  ribsArr[elem_local[0]].a.X);
384             var nu = (y - ribsArr[elem_local[4]].a.Y) / (ribsArr[elem_local[4]].b.Y -
                ↪  ribsArr[elem_local[4]].a.Y);
385             var khi = (z - ribsArr[elem_local[8]].a.Z) / (ribsArr[elem_local[8]].b.Z -
                ↪  ribsArr[elem_local[8]].a.Z);
386
387             double[] q = [Solutions[0][elem_local[0]], Solutions[0][elem_local[1]],
                ↪  Solutions[0][elem_local[2]], Solutions[0][elem_local[3]],
388                           Solutions[0][elem_local[4]], Solutions[0][elem_local[5]],
                               ↪  Solutions[0][elem_local[6]], Solutions[0][elem_local[7]],
389                           Solutions[0][elem_local[8]], Solutions[0][elem_local[9]],
                               ↪  Solutions[0][elem_local[10]], Solutions[0][elem_local[11]]];
390
391             var ans = BasisFunctions3DVec.GetValue(eps, nu, khi, q);
392             var theorValue = Function.A(x, y, z, 0.0D);
393
394             sw.WriteLine($"FEM A  {ans.Item1:E15} {ans.Item2:E15} {ans.Item3:E15}");
```

```
395             sw.WriteLine($"Theor  {theorValue.Item1:E15} {theorValue.Item2:E15}
                ↪  {theorValue.Item3:E15}");
396
397             var currAbsDiscX = Math.Abs(ans.Item1 - theorValue.Item1);
398             var currAbsDiscY = Math.Abs(ans.Item2 - theorValue.Item2);
399             var currAbsDiscZ = Math.Abs(ans.Item3 - theorValue.Item3);
400
401             squareDiffX += currAbsDiscX * currAbsDiscX;
402             squareDiffY += currAbsDiscY * currAbsDiscY;
403             squareDiffZ += currAbsDiscZ * currAbsDiscZ;
404
405             var currRelDiscX = currAbsDiscX / Math.Abs(theorValue.Item1);
406             var currRelDiscY = currAbsDiscY / Math.Abs(theorValue.Item2);
407             var currRelDiscZ = currAbsDiscZ / Math.Abs(theorValue.Item3);
408
409             sw.WriteLine($"CurrAD {currAbsDiscX:E15} {currAbsDiscY:E15}
                ↪  {currAbsDiscZ:E15}");
410             sw.WriteLine($"CurrRD {currRelDiscX:E15} {currRelDiscY:E15}
                ↪  {currRelDiscZ:E15}\n");
411
412             absDiscX += currAbsDiscX;
413             absDiscY += currAbsDiscY;
414             absDiscZ += currAbsDiscZ;
415
416             absDivX += theorValue.Item1;
417             absDivY += theorValue.Item2;
418             absDivZ += theorValue.Item3;
419
420             relDiscX += currRelDiscX * currRelDiscX;
421             relDiscY += currRelDiscY * currRelDiscY;
422             relDiscZ += currRelDiscZ * currRelDiscZ;
423
424             relDivX += theorValue.Item1 * theorValue.Item1;
425             relDivY += theorValue.Item2 * theorValue.Item2;
426             relDivZ += theorValue.Item3 * theorValue.Item3;
427
428             iter++;
429         }
430         sw.WriteLine($"Avg disc: {absDiscX / iter:E15} {absDiscY / iter:E15} {absDiscZ /
            ↪  iter:E15}");
```

```
431         sw.WriteLine($"Rel disc: {Math.Sqrt(relDiscX / relDivX):E15} {Math.Sqrt(relDiscY
            ↪  / relDivY):E15} {Math.Sqrt(relDiscZ / relDivZ):E15}");
432         sw.WriteLine($"SKO: {Math.Sqrt(squareDiffX / elemsArr.Length):E15}
            ↪  {Math.Sqrt(squareDiffY / elemsArr.Length):E15} {Math.Sqrt(squareDiffZ /
            ↪  elemsArr.Length):E15}");
433         sw.Close();
434     }
435 }
```

LocalMatrix3D.cs

```
1  namespace MathObjects;
2
3  public class LocalMatrixG3D (double mu, double hx, double hy, double hz) : Matrix
4  {
5      private readonly double _mu = mu;
6      private readonly double _hx = hx;
7      private readonly double _hy = hy;
8      private readonly double _hz = hz;
9
10     private readonly double[,] G1 = {{ 2.0D,  1.0D, -2.0D, -1.0D},
11                                      { 1.0D,  2.0D, -1.0D, -2.0D},
12                                      {-2.0D, -1.0D,  2.0D,  1.0D},
13                                      {-1.0D, -2.0D,  1.0D,  2.0D}};
14
15     private readonly double[,] G2 = {{ 2.0D, -2.0D,  1.0D, -1.0D},
16                                      {-2.0D,  2.0D, -1.0D,  1.0D},
17                                      { 1.0D, -1.0D,  2.0D, -2.0D},
18                                      {-1.0D,  1.0D, -2.0D,  2.0D}};
19
20     private readonly double[,] G3 = {{-2.0D,  2.0D, -1.0D,  1.0D},
21                                      {-1.0D,  1.0D, -2.0D,  2.0D},
22                                      { 2.0D, -2.0D,  1.0D, -1.0D},
23                                      { 1.0D, -1.0D,  2.0D, -2.0D}};
24
25     public override double this[int i, int j]
26     {
27         get
28         {
29             return (i / 4, j / 4)
```

```
            switch
            {
                (0, 0) => (_hx * _hy / (6.0D * _hz) * G1[i % 4, j % 4] + _hx * _hz /
                ↪  (6.0D * _hy) * G2[i % 4, j % 4]) / _mu,
                (0, 1) or (1, 0) => -1.0D * (_hz / 6.0D) * G2[i % 4, j % 4] / _mu,
                (0, 2) => _hy / 6.0D * G3[i % 4, j % 4] / _mu,
                (1, 1) => (_hx * _hy / (6.0D * _hz) * G1[i % 4, j % 4] + _hy * _hz /
                ↪  (6.0D * _hx) * G2[i % 4, j % 4]) / _mu,
                (1, 2) or (2, 1) => -1.0D * _hx / 6.0D * G1[i % 4, j % 4] / _mu,
                (2, 0) => _hy / 6.0D * G3[j % 4, i % 4] / _mu,
                (2, 2) => (_hx * _hz / (6.0D * _hy) * G1[i % 4, j % 4] + _hy * _hz /
                ↪  (6.0D * _hx) * G2[i % 4, j % 4]) / _mu,
                _ => throw new ArgumentOutOfRangeException("Out of local matrix 3d
                ↪  range"),
            };
        }
        set{}
    }
}

public class LocalMatrixM3D(double gamma, double hx, double hy, double hz) : Matrix
{
    private readonly double _gamma = gamma;
    private readonly double _hx = hx;
    private readonly double _hy = hy;
    private readonly double _hz = hz;

    private readonly double[,] D = {{4.0D, 2.0D, 2.0D, 1.0D},
                                    {2.0D, 4.0D, 1.0D, 2.0D},
                                    {2.0D, 1.0D, 4.0D, 2.0D},
                                    {1.0D, 2.0D, 2.0D, 4.0D}};

    public override double this[int i, int j]
    {
        get
        {
            return (i / 4, j / 4)
            switch
            {
```

```
                (0, 0) or (1, 1) or (2, 2) => _gamma * _hx * _hy * _hz / 36.0D * D[i % 4,
                ↪  j % 4],
                (0, 1) or (0, 2) or (1, 0) or (1, 2) or (2, 0) or (2, 1) => 0.0D,
                _ => throw new ArgumentOutOfRangeException("Out of local matrix 3d
                ↪  range"),
            };
        }
        set{}
    }
}
```

LocalVector3D.cs

```
using static Functions.Function;

namespace MathObjects;


public class LocalVector3D : Vector
{
    private readonly double x0;
    private readonly double x1;
    private readonly double y0;
    private readonly double y1;
    private readonly double z0;
    private readonly double z1;
    private readonly double xm;
    private readonly double ym;
    private readonly double zm;
    private readonly double t;

    private LocalMatrixM3D M;

    public override double this[int i]
    {
        get
        {
            double ScalarMult((double, double, double) a, (double, double, double) b)
            => a.Item1 * b.Item1 + a.Item2 * b.Item2 + a.Item3 * b.Item3;

```

```
            (double, double, double) Divide ((double, double, double) v, double sc)
            => (v.Item1 / sc, v.Item2 / sc, v.Item3 / sc);

            double Length((double, double, double) v)
            => Math.Sqrt(Math.Pow(v.Item1, 2) + Math.Pow(v.Item2, 2) + Math.Pow(v.Item3,
            ↪  2));

            // Очень спорно.
            List<double> vect = [
                ScalarMult(F(xm, y0, z0, t), (1.0D, 0.0D, 0.0D)),
                ScalarMult(F(xm, y1, z0, t), (1.0D, 0.0D, 0.0D)),
                ScalarMult(F(xm, y0, z1, t), (1.0D, 0.0D, 0.0D)),
                ScalarMult(F(xm, y1, z1, t), (1.0D, 0.0D, 0.0D)),

                ScalarMult(F(x0, ym, z0, t), (0.0D, 1.0D, 0.0D)),
                ScalarMult(F(x1, ym, z0, t), (0.0D, 1.0D, 0.0D)),
                ScalarMult(F(x0, ym, z1, t), (0.0D, 1.0D, 0.0D)),
                ScalarMult(F(x1, ym, z1, t), (0.0D, 1.0D, 0.0D)),

                ScalarMult(F(x0, y0, zm, t), (0.0D, 0.0D, 1.0D)),
                ScalarMult(F(x1, y0, zm, t), (0.0D, 0.0D, 1.0D)),
                ScalarMult(F(x0, y1, zm, t), (0.0D, 0.0D, 1.0D)),
                ScalarMult(F(x1, y1, zm, t), (0.0D, 0.0D, 1.0D))
            ];

            double ans = 0.0D;
            for (int j = 0; j < vect.Count; j++)
                ans += M[i, j] * vect[j];
            return ans;
        }
    }

    public LocalVector3D(double x0, double x1, double y0, double y1, double z0, double
    ↪  z1, double t)
    {
        this.x0 = x0;
        this.x1 = x1;
        this.y0 = y0;
        this.y1 = y1;
        this.z0 = z0;
```

```
66          this.z1 = z1;
67          this.t = t;
68          xm = 0.5D * (x1 + x0);
69          ym = 0.5D * (y1 + y0);
70          zm = 0.5D * (z1 + z0);
71          M = new(1.0, x1 - x0, y1 - y0, z1 - z0);
72          Generate();
73      }
74
75      private void Generate()
76      {
77          double ScalarMult((double, double, double) a, (double, double, double) b)
78          => a.Item1 * b.Item1 + a.Item2 * b.Item2 + a.Item3 * b.Item3;
79
80          List<double> vect = [
81              ScalarMult(F(xm, y0, z0, t), (1.0D, 0.0D, 0.0D)),
82              ScalarMult(F(xm, y1, z0, t), (1.0D, 0.0D, 0.0D)),
83              ScalarMult(F(xm, y0, z1, t), (1.0D, 0.0D, 0.0D)),
84              ScalarMult(F(xm, y1, z1, t), (1.0D, 0.0D, 0.0D)),
85
86              ScalarMult(F(x0, ym, z0, t), (0.0D, 1.0D, 0.0D)),
87              ScalarMult(F(x1, ym, z0, t), (0.0D, 1.0D, 0.0D)),
88              ScalarMult(F(x0, ym, z1, t), (0.0D, 1.0D, 0.0D)),
89              ScalarMult(F(x1, ym, z1, t), (0.0D, 1.0D, 0.0D)),
90
91              ScalarMult(F(x0, y0, zm, t), (0.0D, 0.0D, 1.0D)),
92              ScalarMult(F(x1, y0, zm, t), (0.0D, 0.0D, 1.0D)),
93              ScalarMult(F(x0, y1, zm, t), (0.0D, 0.0D, 1.0D)),
94              ScalarMult(F(x1, y1, zm, t), (0.0D, 0.0D, 1.0D))
95          ];
96
97          double ans = 0.0D;
98          for (int i = 0; i < vect.Count; i++)
99          {
100             for (int j = 0; j < vect.Count; j++)
101                 ans += M[i, j] * vect[j];
102             ans = 0.0D;
103         }
104     }
105 }
```

FEM3D.cs

```
namespace Project;

using System.Collections.Immutable;
using System.Numerics;
using MathObjects;
using Solver;
using Grid;
using DataStructs;
using System.Diagnostics;
using Functions;
using System.ComponentModel.DataAnnotations;
using System.Timers;

public class FEM3D : FEM
{
    public ArrayOfRibs ribsArr;

    // Maybe private?
    public List<Layer> Layers;

    public FEM3D()
    {
        Layers = [];
        mesh = new Mesh3Dim
        {
            nodesX = [],
            nodesY = [],
            nodesZ = []
        };
    }

    public void ConstructMesh(int nx, int ny, int nz)
    {
        if (mesh == null) throw new ArgumentNullException("Mesh is null");

        double hx = 2.0D;
        double hy = 2.0D;
        double hz = 2.0D;
```

```
39
40          mesh.nodesX = [];
41          mesh.nodesY = [];
42          mesh.nodesZ = [];
43
44          for (int i = 0; i < nx; i++)
45          {
46              double stepx = hx / (nx - 1);
47              mesh.nodesX.Add(i * stepx);
48          }
49
50          for (int i = 0; i < ny; i++)
51          {
52              double stepy = hy / (ny - 1);
53              mesh.nodesY.Add(i * stepy);
54          }
55
56          for (int i = 0; i < nz; i++)
57          {
58              double stepz = hz / (nz - 1);
59              mesh.nodesZ.Add(i * stepz);
60          }
61
62          timeMesh = [1.0D];
63      }
64
65
66
67      public void GenerateArrays()
68      {
69          if (mesh is null) throw new ArgumentNullException("mesh is null!");
70          pointsArr = MeshGenerator.GenerateListOfPoints(mesh);
71          ribsArr = MeshGenerator.GenerateListOfRibs(mesh, pointsArr);
72          elemsArr = MeshGenerator.GenerateListOfElems(mesh, ribsArr);
73          bordersArr = MeshGenerator.GenerateListOfBorders(mesh);
74      }
75
76      public void AddField(Layer layer)
77      {
78          ArgumentNullException.ThrowIfNull(layer);
```

```
79          Layers.Add(layer);
80      }
81
82      public void CommitFields()
83      {
84          if (elemsArr is null) throw new ArgumentNullException("elemsArr is null");
85
86          foreach (var layer in Layers)
87          {
88              for (int i = 0; i < elemsArr.Length; i++)
89              {
90                  double minz = Math.Min(ribsArr[elemsArr[i][^1]].a.Z,
                    ↪  ribsArr[elemsArr[i][^1]].b.Z);
91                  double maxz = Math.Max(ribsArr[elemsArr[i][^1]].a.Z,
                    ↪  ribsArr[elemsArr[i][^1]].b.Z);
92                  if (layer.z0 <= minz && maxz <= layer.z1)
93                      elemsArr.sigmai[i] = layer.sigma;
94              }
95          }
96      }
97
98      public void ConstructMatrixAndVector()
99      {
100         if (elemsArr is null) throw new ArgumentNullException("elemsArr is null");
101         if (bordersArr is null) throw new ArgumentNullException("bordersArr is null");
102
103         var sparceMatrix = new GlobalMatrix(ribsArr.Count);
104         Generator.BuildPortait(ref sparceMatrix, ribsArr.Count, elemsArr);
105
106         var G = new GlobalMatrix(sparceMatrix);
107         Generator.FillMatrixG(ref G, ribsArr, elemsArr);
108
109         var M = new GlobalMatrix(sparceMatrix);
110         Generator.FillMatrixM(ref M, ribsArr, elemsArr);
111
112         Matrix = G + M;
113
114         var b = new GlobalVector(ribsArr.Count);
115         Generator.FillVector3D(ref b, ribsArr, elemsArr, 0.0);
116         Vector = b;
```

```
117
118         Generator.ConsiderBoundaryConditions(ref Matrix, ref Vector, ribsArr, bordersArr,
                ↪ 0.0D);
119     }
120
121     public void Solve()
122     {
123         if (solver is null) throw new ArgumentNullException("Solver is null");
124         if (Matrix is null) throw new ArgumentNullException("Matrix is null");
125         if (Vector is null) throw new ArgumentNullException("Vector is null");
126         Solutions = new GlobalVector[timeMesh.Length];
127         Discrepancy = new GlobalVector[timeMesh.Length];
128         (Solutions[0], Discrepancy[0]) = solver.Solve(Matrix, Vector);
129     }
130
131     public void TestOutput(string path)
132     {
133         using var sw = new StreamWriter(path + "/Answer3D/Answer_Test.txt");
134
135         var absDiscX = 0.0D;
136         var absDiscY = 0.0D;
137         var absDiscZ = 0.0D;
138
139         var absDivX = 0.0D;
140         var absDivY = 0.0D;
141         var absDivZ = 0.0D;
142
143         var relDiscX = 0.0D;
144         var relDiscY = 0.0D;
145         var relDiscZ = 0.0D;
146
147         var relDivX = 0.0D;
148         var relDivY = 0.0D;
149         var relDivZ = 0.0D;
150
151         var squareDiffX = 0.0D;
152         var squareDiffY = 0.0D;
153         var squareDiffZ = 0.0D;
154
155         int iter = 0;
```

```
156
157         foreach (var elem in elemsArr)
158         {
159             int[] elem_local = [elem[0], elem[3], elem[8], elem[11],
160                                 elem[1], elem[2], elem[9], elem[10],
161                                 elem[4], elem[5], elem[6], elem[7]];
162
163             var x = 0.5D * (ribsArr[elem_local[0]].a.X + ribsArr[elem_local[0]].b.X);
164             var y = 0.5D * (ribsArr[elem_local[4]].a.Y + ribsArr[elem_local[4]].b.Y);
165             var z = 0.5D * (ribsArr[elem_local[8]].a.Z + ribsArr[elem_local[8]].b.Z);
166
167             sw.WriteLine($"Points {x:E15} {y:E15} {z:E15}");
168
169             var eps = (x - ribsArr[elem_local[0]].a.X) / (ribsArr[elem_local[0]].b.X -
                ↪  ribsArr[elem_local[0]].a.X);
170             var nu =  (y - ribsArr[elem_local[4]].a.Y) / (ribsArr[elem_local[4]].b.Y -
                ↪  ribsArr[elem_local[4]].a.Y);
171             var khi = (z - ribsArr[elem_local[8]].a.Z) / (ribsArr[elem_local[8]].b.Z -
                ↪  ribsArr[elem_local[8]].a.Z);
172
173             double[] q = [Solutions[0][elem_local[0]], Solutions[0][elem_local[1]],
                ↪  Solutions[0][elem_local[2]], Solutions[0][elem_local[3]],
174                           Solutions[0][elem_local[4]], Solutions[0][elem_local[5]],
                               ↪  Solutions[0][elem_local[6]], Solutions[0][elem_local[7]],
175                           Solutions[0][elem_local[8]], Solutions[0][elem_local[9]],
                               ↪  Solutions[0][elem_local[10]], Solutions[0][elem_local[11]]];
176
177             var ans = BasisFunctions3DVec.GetValue(eps, nu, khi, q);
178             var theorValue = Function.A(x, y, z, 0.0D);
179
180             sw.WriteLine($"FEM A  {ans.Item1:E15} {ans.Item2:E15} {ans.Item3:E15}");
181             sw.WriteLine($"Theor  {theorValue.Item1:E15} {theorValue.Item2:E15}
                ↪  {theorValue.Item3:E15}");
182
183             var currAbsDiscX = Math.Abs(ans.Item1 - theorValue.Item1);
184             var currAbsDiscY = Math.Abs(ans.Item2 - theorValue.Item2);
185             var currAbsDiscZ = Math.Abs(ans.Item3 - theorValue.Item3);
186
187             squareDiffX += currAbsDiscX * currAbsDiscX;
188             squareDiffY += currAbsDiscY * currAbsDiscY;
```

```
189             squareDiffZ += currAbsDiscZ * currAbsDiscZ;
190
191             var currRelDiscX = currAbsDiscX / Math.Abs(theorValue.Item1);
192             var currRelDiscY = currAbsDiscY / Math.Abs(theorValue.Item2);
193             var currRelDiscZ = currAbsDiscZ / Math.Abs(theorValue.Item3);
194
195             sw.WriteLine($"CurrAD {currAbsDiscX:E15} {currAbsDiscY:E15}
                ↪  {currAbsDiscZ:E15}");
196             sw.WriteLine($"CurrRD {currRelDiscX:E15} {currRelDiscY:E15}
                ↪  {currRelDiscZ:E15}\n");
197
198             absDiscX += currAbsDiscX;
199             absDiscY += currAbsDiscY;
200             absDiscZ += currAbsDiscZ;
201
202             absDivX += theorValue.Item1;
203             absDivY += theorValue.Item2;
204             absDivZ += theorValue.Item3;
205
206             relDiscX += currRelDiscX * currRelDiscX;
207             relDiscY += currRelDiscY * currRelDiscY;
208             relDiscZ += currRelDiscZ * currRelDiscZ;
209
210             relDivX += theorValue.Item1 * theorValue.Item1;
211             relDivY += theorValue.Item2 * theorValue.Item2;
212             relDivZ += theorValue.Item3 * theorValue.Item3;
213
214             iter++;
215         }
216         sw.WriteLine($"Avg disc: {absDiscX / iter:E15} {absDiscY / iter:E15} {absDiscZ /
            ↪  iter:E15}");
217         sw.WriteLine($"Rel disc: {Math.Sqrt(relDiscX / relDivX):E15} {Math.Sqrt(relDiscY
            ↪  / relDivY):E15} {Math.Sqrt(relDiscZ / relDivZ):E15}");
218         sw.WriteLine($"SKO: {Math.Sqrt(squareDiffX / elemsArr.Length):E15}
            ↪  {Math.Sqrt(squareDiffY / elemsArr.Length):E15} {Math.Sqrt(squareDiffZ /
            ↪  elemsArr.Length):E15}");
219         sw.Close();
220     }
221 }
```